

Autonomous Claims Orchestrator

End-to-End Workflow Documentation

Tech Stacks · Microservices · LLM Models · Data Flows

Backend	Python 3 · FastAPI · Uvicorn · Pydantic v2
Frontend	Next.js 16 · React 18 · TypeScript · Tailwind · Recharts
LLM (extraction)	gpt-4o (email, document, vision)
LLM (classification)	gpt-4o-mini (FNOL gate, FAQ detection, FAQ answer)
Email ingest	IMAP4_SSL (imap.gmail.com:993) + SendGrid Inbound Parse webhook
Storage	JSON file store + CSV (no SQL at runtime)
Notifications	smtplib SMTP (FNOL ack, under-review, desk-rejection)
Microservices	email_ingestion · ingested_claims · extraction · decision · faq_resolution · process_claim · dashboard

1. End-to-End Workflow Overview

Step	Phase	Service	Tech Stack	LLM
1	Email Ingestion	email_ingestion	Python stdlib · imaplib · ssl · email · IMAP4_SSL	gpt-4o-mini (FNOL classifier, yes/no, max_tokens=10, temp=0)
2	SendGrid Inbound (Alternate Ingestion)	frontend webhook + ingested_claims	Next.js API Route · SendGrid Inbound Parse · multipart/form-data	None
3	Ingested Claims Storage	ingested_claims	Python · JSON file storage · regex · stdlib	None
4	Claim Extraction	extraction	Python · OpenAI Python SDK · base64 · mimetypes	gpt-4o (email extraction, document extraction, vision) · temp=0.1 · max_tokens=1500 (email/doc) / 2500 (vision)
5	Policy Grounding (Local DB)	decision / policy_grounding_local	Python · JSON file DB · regex · rule-based scoring	None
6	Policy Grounding (ISO Clause Fallback)	decision / policy_clauses	Python · cosine/keyword similarity · static ISO clause library	None
7	Decision Pack Assembly	decision	Python · rule-based logic · Pydantic-style dicts	None (extraction already ran LLM; decision is deterministic)
8	Process Claim Orchestration	process_claim	Python · rule-based orchestration	Indirect: delegates to extraction (gpt-4o) and FAQ (gpt-4o-mini)
9	Dashboard & Processed Claims Storage	dashboard	Python · JSON · CSV · file I/O	None
10	Claim Notifications (SMTP)	claim_notification	Python · smtplib · ssl · SMTP/SMTPS	None
11	FAQ Resolution	faq_resolution	Python · OpenAI SDK · CSV · smtplib · regex	gpt-4o-mini (FAQ vs CLAIM classification + FAQ answer index selection)
12	FastAPI REST Layer	fastapi_server	Python · FastAPI · Uvicorn · Pydantic v2 · CORS middleware	None (delegates to services)
13	Frontend – Next.js Application	frontend	Next.js 16 · React 18 · TypeScript · Tailwind CSS · Recharts · OpenAI npm · LangChain/LangGraph (deps present, main flow uses FastAPI)	N/A (frontend calls FastAPI; LangGraph client-side orchestrator present but not used in main claim flow)

2. Detailed Step-by-Step Workflow

Step 1 | Email Ingestion

Service / Module	email_ingestion
File(s)	backend/email_ingestion/service.py
Tech Stack	Python stdlib · imaplib · ssl · email · IMAP4_SSL
LLM Model	gpt-4o-mini (FNOL classifier, yes/no, max_tokens=10, temp=0)
Output	Saves to data/ingested-claims.json via save_ingested_claim(); attachments stored in data/ingested-attachments/{claimId}/; triggers send_fnol_received_acknowledgement_email() (SMTP) if FNOL_RECEIPT_ACK_EMAIL_ENABLED=true

What Happens (Step by Step):

- Load env vars via `_load_env()` from `ENV_FILE` (.env)
- Open IMAP4_SSL connection (default: imap.gmail.com:993, SSL verify OFF)
- Select mailbox: `IMAP_MAILBOX` env (default INBOX)
- SEARCH ALL (if `IMAP_SYNC_INCLUDE_READ=true`) or UNSEEN
- Gmail fallback: tries [Gmail]/All Mail → [Google Mail]/All Mail → INBOX
- Fetch each email as RFC822 raw bytes
- Parse MIME: extract subject, from, to, date, Message-ID, plain body, attachments
- Body extraction: prefers text/plain; fallback text/html (HTML stripped via `_strip_html`)

Filters Applied:

- Duplicate check: Message-ID OR subject[from]date hash
- Thread-reply fast-path: if reply to existing claim → merge, skip FNOL gate
- `_should_reject_consumer_complaint_correspondence`: OEM/retail complaint patterns, short replies on quoted threads
- `_is_clearly_procedural_faq_question`: 'how do I file', policy expiry questions → route to FAQ, not FNOL
- `_has_relevant_keywords` / `_has_strong_keywords`: applied on latest non-quoted segment (`_primary_message_text`)
- LLM FNOL gate (gpt-4o-mini): system='You classify P&C; emails – answer only yes or no. Is this FNOL?'; bypass if `FNOL_FILTER_ENABLED=false` or strong-keyword path + no API key

Policy Number Handling:

Regex in `ingested_claims/service.py` `extract_policy_number()`; patterns: keyword-anchored (`poli?cy\s*(?:number[no]#...)\s*[:#-]=\s*([A-Z0-9]{4,}...)`), structural (`(\b[A-Z]{2}\d{6,}\b)`), claim-number fallback; if none found: sender email / Message-ID display used as fallback

Step 2 | SendGrid Inbound (Alternate Ingestion)

Service / Module	frontend webhook + ingested_claims
File(s)	frontend/app/api/webhooks/sendgrid-inbound/route.ts · backend/ingested_claims/service.py
Tech Stack	Next.js API Route · SendGrid Inbound Parse · multipart/form-data
LLM Model	None
Output	Same JSON storage + FNOL ack email as IMAP path

What Happens (Step by Step):

- SendGrid posts multipart form-data to `/api/webhooks/sendgrid-inbound`
- Next.js route parses fields: from, to, subject, text, html, attachments
- If body is empty, `extract_plain_body_from_rfc822()` parses raw email field

- Calls POST /api/ingested-claims on FastAPI (Python backend) with same payload shape as IMAP path
- Python backend deduplicates and stores via same save_ingested_claim() function

Filters Applied:

Same dedup + LLM FNOL gate as IMAP path via shared ingested_claims/service.py logic

Policy Number Handling:

Same extract_policy_number() regex as IMAP path

Step 3 | Ingested Claims Storage

Service / Module	ingested_claims
File(s)	backend/ingested_claims/service.py · mail_chain.py
Tech Stack	Python · JSON file storage · regex · stdlib
LLM Model	None
Output	data/ingested-claims.json, data/ingested-attachments/{claimId}/*

What Happens (Step by Step):

- save_ingested_claim(): writes/updates data/ingested-claims.json
- Thread merging: mail_chain.py groups email chains by subject / policy number
- Dedup: is_duplicate_email() checks Message-ID first, then subject|from|date fingerprint
- get_ingested_claim_by_id(): lookup by claimId
- get_all_ingested_claims(): list with optional source filter (faq / fnol)
- clear_all_ingested_claims(): wipe for dev/demo reset
- extract_policy_number() applied at save time; stored in claim.policyNumber

Filters Applied:

N/A – storage layer

Policy Number Handling:

Stored at ingest time from regex extraction

Step 4 | Claim Extraction

Service / Module	extraction
File(s)	backend/extraction/service.py
Tech Stack	Python · OpenAI Python SDK · base64 · mimetypes
LLM Model	gpt-4o (email extraction, document extraction, vision) · temp=0.1 · max_tokens=1500 (email/doc) / 2500 (vision)
Output	Dict: extractedFields{}, evidence[], documentAnalysis[]; passed directly to build_decision_pack()

What Happens (Step by Step):

- Entry: extract_claim_information(claim_id, email_body, attachments)
- Step A – Email extraction: extract_from_email(email_body) → JSON schema prompt
- System: 'You are an expert insurance claims processor. Extract JSON only.'
- Fields: policyNumber, claimantName, contactEmail, contactPhone, lossDate,
- lossType (fire/water/auto/theft/liability/medical/other),
- lossLocation, description, vehicleInfo, propertyAddress,
- estimatedAmount, _confidence (per-field 0.0–1.0)
- Step B – Attachment processing (per attachment):

- Image files → analyze_image_with_vision(): multimodal message with image_url data URL
- Prompt: structured KPIs (damage severity, repair cost, injury indicators) + detailed_summary
- Text/PDF → extract_from_document(): classify type + extract keyFields
- Document types: PoliceReport, RepairEstimate, Invoice, MedicalRecord, IncidentReport, DamagePhoto, Other
- Binary/unknown → placeholder entry
- Step C – Evidence list: builds evidence[] array from scalar field values + confidence scores
- Returns: { extractedFields, evidence, documentAnalysis[] }

Policy Number Handling:

LLM extracts policyNumber from email body (re-extraction with full context, higher accuracy than regex)

Step 5 | Policy Grounding (Local DB)

Service / Module	decision / policy_grounding_local
File(s)	backend/decision/policy_grounding_local.py · database/local_data/*.json
Tech Stack	Python · JSON file DB · regex · rule-based scoring
LLM Model	None
Output	grounding result dict; if empty → fallback to policy_clauses.py similarity engine

What Happens (Step by Step):

- Input: policyNumber (from LLM extraction) OR sender email
- find_customer_by_email(): email → customer record from customers.json
- find_policy_by_number(): policyNumber → policy record from policies.json
- Policy active check: status == 'active' + expiry date >= today
- Coverage code lookup: policy_grounding_mapping.json maps lossType × policy_type → covered clause IDs
- policy_type detection: AUTO / HOME / COMMERCIAL from policy.policy_type field
- Per-clause confidence scoring formula:
 - base = 1.0; deduct for inactive policy, expiry within 30 days, lossType mismatch;
 - sort descending, return top clauses
- build_policy_holder_info(): assembles name, address, contact from customer + policy records (PII masked in draft)
- Returns: { groundingClauses[], coverageConfirmed, policyStatus, policyHolderInfo }

Policy Number Handling:

Resolves policyNumber → full policy record

Step 6 | Policy Grounding (ISO Clause Fallback)

Service / Module	decision / policy_clauses
File(s)	backend/decision/policy_clauses.py
Tech Stack	Python · cosine/keyword similarity · static ISO clause library
LLM Model	None
Output	groundingClauses[] with similarity scores; used as fallback in build_decision_pack()

What Happens (Step by Step):

- POLICY_CLAUSES: static list of ISO PAP / HO-3 / CGL-style clause texts
- _infer_product_types(): from policy number prefix (AC→auto, HO→home, CL→commercial)
- _infer_loss_types(): regex on lossType + description text
- _compute_similarity(): keyword overlap between clause text and claim description
- Threshold: CONFIDENCE_THRESHOLD_HIGH=0.8, MEDIUM=0.6

- Only clauses with similarity ≥ 0.6 returned; max 6 results
- `get_policy_grounding()`: wraps above steps, returns same shape as local grounding

Filters Applied:

similarity < 0.6 → excluded

Policy Number Handling:

N/A – uses already-extracted policy data

Step 7 | Decision Pack Assembly

Service / Module	decision
File(s)	backend/decision/service.py
Tech Stack	Python · rule-based logic · Pydantic-style dicts
LLM Model	None (extraction already ran LLM; decision is deterministic)
Output	decisionPack dict; consumed by process_claim orchestrator

What Happens (Step by Step):

- `build_decision_pack(claim_id, extraction_result)`
- Merge `extractedFields` from email extraction
- Resolve policy: use LLM-extracted `policyNumber` OR fall back via sender email lookup
- Run `get_policy_grounding_from_local_data()`; if empty → `get_policy_grounding()` (ISO clauses)
- Derive `coverage_confirmed`: True if grounding returned clauses + policy is active
- Policy status flags: POLICY-EXPIRED / POLICY-INACTIVE / POLICY-NOT-FOUND / CUSTOMER-NOT-FOUND
- Build `policy_holder_info` from local DB (name, address, policy limits, deductible)
- Assemble `decisionPack`:
 - `claimId`, `timestamp`, `processingStatus`
 - `extractedFields` (with `_confidence` map)
 - `evidence[]` list
 - `documentAnalysis[]` (attachment results)
 - `groundingClauses[]` (applicable policy clauses)
 - `coverageConfirmed` bool
 - `policyHolderInfo`
 - `auditMetadata` (model used, processing time, data sources)

Filters Applied:

Policy status → auto-reject flags if EXPIRED/INACTIVE/NOT-FOUND

Policy Number Handling:

Final resolution: LLM extraction → DB lookup → grounding

Step 8 | Process Claim Orchestration

Service / Module	process_claim
File(s)	backend/process_claim/orchestrator.py
Tech Stack	Python · rule-based orchestration
LLM Model	Indirect: delegates to extraction (gpt-4o) and FAQ (gpt-4o-mini)
Output	Processed claim JSON saved; email notification sent to claimant

What Happens (Step by Step):

- `POST /api/process-claim` → FastAPI → `process_claim(claim_id)`

- 1. `get_ingested_claim_by_id(claim_id)` – load raw ingested claim
- 2. `extract_claim_information()` – run extraction pipeline
- 3. `build_decision_pack()` – run grounding + decision assembly
- 4. Auto-reject check: if `policyStatus` in `[POLICY-EXPIRED, POLICY-INACTIVE, POLICY-NOT-FOUND, CUSTOMER-NOT-FOUND]`
- AND `policyHolderInfo.status == EXPIRED` → `send_desk_rejection_email()`
- → `status = 'desk_rejected'` → `save_processed_claim()`
- 5. Otherwise: `status = 'under_review'` → `save_processed_claim()`
- → triggers `send_claim_under_review_email()` from `dashboard/service.py`

Filters Applied:

Auto-reject on invalid/expired policy conditions

Policy Number Handling:

Uses final resolved `policyNumber` from step 7

Step 9 | Dashboard & Processed Claims Storage

Service / Module	dashboard
File(s)	backend/dashboard/service.py
Tech Stack	Python · JSON · CSV · file I/O
LLM Model	None
Output	data/processed-claims/*.json, claims-index.json, claims-history.csv; KPI JSON for frontend

What Happens (Step by Step):

- `save_processed_claim()`: write `data/processed-claims/{claimId}.json`
- Update `data/processed-claims/claims-index.json`
- Append row to `data/processed-claims/claims-history.csv`
- If new claim (not `desk_rejected`): call `send_claim_under_review_email()`
- `get_dashboard_kpis()`: reads all claim JSONs + index
- KPIs computed: total claims, claims by date, claims by lossType,
- coverage breakdown, evidence confidence distribution,
- document type counts, desk-rejection rate
- `clear_all_processed_claims()`: wipe for dev/demo

Step 10 | Claim Notifications (SMTP)

Service / Module	claim_notification
File(s)	backend/claim_notification/service.py
Tech Stack	Python · smtplib · ssl · SMTP/SMTPS
LLM Model	None
Output	Outbound SMTP emails to claimant; threaded to original claim email

What Happens (Step by Step):

- 1. FNOL Receipt Acknowledgement (`send_fnol_received_acknowledgement_email`)
- Triggered: after successful IMAP / SendGrid ingest
- Toggle: `FNOL_RECEIPT_ACK_EMAIL_ENABLED` (default: on)
- Content: 'We received your claim, reference #{claimId}'
- 2. Claim Under Review (`send_claim_under_review_email`)
- Triggered: after `save_processed_claim()` for non-desk-rejected claims

- Toggle: CLAIM_UNDER_REVIEW_EMAIL_ENABLED (default: on)
- Content: claim details, policy holder info, next steps
- 3. Desk Rejection (send_desk_rejection_email)
- Triggered: orchestrator detects expired/inactive/not-found policy
- Toggle: DESK_REJECTION_EMAIL_ENABLED (default: on)
- Content: rejection reason, policy status, appeal instructions
- SMTP config: SENDER_EMAIL, EMAIL_PASSWORD, SMTP_HOST, SMTP_PORT, SMTP_SECURE
- Reply threading: sets In-Reply-To / References headers matching original email

Step 11 | FAQ Resolution

Service / Module	faq_resolution
File(s)	backend/faq_resolution/service.py
Tech Stack	Python · OpenAI SDK · CSV · smtplib · regex
LLM Model	gpt-4o-mini (FAQ vs CLAIM classification + FAQ answer index selection)
Output	SMTP reply sent to sender; message ID stored to prevent duplicate replies

What Happens (Step by Step):

- process_faq_email(from, to, subject, body, message_id)
- 1. Dedup: check faq-answered-ids.json + message_id → skip if already answered
- 2. _is_faq_query(): regex layer (keyword patterns) + LLM ('FAQ or CLAIM? answer one word')
- 3. If FAQ detected and SMTP configured:
 - a. _classify_customer_data_intent(): detect 'what is my policy limit / deductible / status'
 - → _answer_customer_data_query(): look up customer/policy JSON DB → personalised reply
 - b. Else: _find_faq_answer(): LLM receives full FAQ.csv context;
 - prompt asks to pick 'Q1', 'Q2', ... matching the question; extract answer text
 - c. Keyword fallback (no API key): overlap score between question and FAQ rows
- 4. _send_faq_response_email(): SMTP threaded reply with In-Reply-To / References headers
- 5. Persist message_id to faq-answered-ids.json to prevent duplicate replies
- FAQ CSV: data/FAQ.csv (question, answer pairs)

Filters Applied:

_is_faq_query() gate; if classified as CLAIM → not handled here (goes to FNOL path)

Policy Number Handling:

Reads policy data from local JSON DB for personalised answers

Step 12 | FastAPI REST Layer

Service / Module	fastapi_server
File(s)	backend/fastapi_server.py
Tech Stack	Python · FastAPI · Uvicorn · Pydantic v2 · CORS middleware
LLM Model	None (delegates to services)
Output	JSON responses to frontend; all mutations persisted by downstream services

What Happens (Step by Step):

- Host: API_HOST (default 0.0.0.0), Port: API_PORT (default 8020)
- Hot reload: API_RELOAD env toggle
- CORS: CORS_ORIGINS (default * – allow all)
- Endpoints:

- GET /health → liveness probe
- POST /api/process-claim → runs orchestrator pipeline
- GET /api/ingested-claims?full=bool&source=faq → list ingested claims
- GET /api/ingested-claims/{claim_id} → single ingested claim
- POST /api/ingested-claims/clear → wipe ingested claims
- POST /api/claims/clear → wipe all processed claims
- GET /api/claims → list processed claims
- POST /api/claims → create/update processed claim
- GET /api/claims/{claim_id} → single processed claim
- GET /api/dashboard/kpis → KPI aggregation
- POST /api/sync-inbox → trigger IMAP sync
- GET / → root health/info

Step 13 | Frontend – Next.js Application

Service / Module	frontend
File(s)	frontend/app/page.tsx · components/*.tsx · lib/backend.ts
Tech Stack	Next.js 16 · React 18 · TypeScript · Tailwind CSS · Recharts · OpenAI npm · LangChain/LangGraph (deps present, main flow uses FastAPI)
LLM Model	N/A (frontend calls FastAPI; LangGraph client-side orchestrator present but not used in main claim flow)
Output	Browser UI; all claim data read/written via FastAPI

What Happens (Step by Step):

- app/page.tsx: stage router – home → review → decision → dashboard; auth gate; FAQ page toggle
- HomePage.tsx: shows ingested claim inbox; IMAP sync via POST /api/sync-inbox; process claim via POST /api/process-claim
- ReviewPage.tsx: displays extracted fields + evidence list for adjuster review
- DecisionPage.tsx: shows decision pack – policy grounding, coverage confirmed, policy holder info
- DashboardPage.tsx: KPI charts via GET /api/dashboard/kpis; uses Recharts for visualisations
- FAQPage.tsx: FAQ-sourced inbox (source=faq query param)
- Header.tsx: navigation between pages
- ClaimSummaryBar.tsx: persistent claim summary across stages
- lib/backend.ts: runPython() helper for legacy webhook spawning
- lib/services/openai.ts: OpenAI client config (NEXT_PUBLIC_OPENAI_API_KEY)
- lib/agents/orchestrator.ts: LangGraph-style client-side orchestrator (not active in main flow)
- frontend/app/api/ingested-claims/route.ts: Next.js proxy route → FastAPI

3. LLM Models – Detailed Usage

Model	Service	Call	Purpose	Parameters	Bypass
gpt-4o	Extraction	email_extraction	Extract structured fields from FNOL email body	temp=0.1, max_tokens=1500	
gpt-4o	Extraction	document_extraction	Classify attachment type and extract key fields	temp=0.1, max_tokens=1500	
gpt-4o	Extraction	vision_analysis	Analyse damage photos, multimodal image_url data URL	max_tokens=2500	
gpt-4o-mini	Email Ingestion	fnol_classifier	Binary yes/no: is this email a P&C; FNOL? Answer only yes or no.	max_tokens=10, temp=0	
gpt-4o-mini	FAQ Resolution	faq_vs_claim	One-word classification: FAQ or CLAIM?	default settings	
gpt-4o-mini	FAQ Resolution	faq_answer_selector	Pick matching Q-index (Q1, Q2, ...) from FAQ.csv context; extract answer text	default settings	

4. Email Ingestion – Deep Dive

4.1 IMAP Configuration

Host (default)	imap.gmail.com [env: IMAP_HOST]
Port (default)	993 [env: IMAP_PORT]
Protocol	IMAP4_SSL; SSL certificate verify OFF by default (IMAP_SSL_VERIFY=false)
Auth	SENDER_EMAIL / IMAP_USER + EMAIL_PASSWORD / IMAP_PASSWORD (spaces automatically stripped)
Mailbox	INBOX [env: IMAP_MAILBOX]; Gmail: tries [Gmail]/All Mail → [Google Mail]/All Mail → INBOX
Fetch scope	SEARCH ALL (IMAP_SYNC_INCLUDE_READ=true, default) or SEARCH UNSEEN
Max emails	IMAP_SYNC_MAX_EMAILS=0 → unlimited

4.2 Filters Applied (Application-Side)

- 1. Duplicate:** Message-ID exact match → skip. Fallback fingerprint: SHA of subject|from|date when no Message-ID.
- 2. Thread reply:** If in-reply-to matches existing claim ID → merge email into thread; skip FNOL re-classification entirely.
- 3. Consumer complaint:** `_should_reject_consumer_complaint_correspondence()`: detects OEM / retail complaint patterns, short replies on quoted threads → classified as non-FNOL, dropped.
- 4. Procedural FAQ:** `_is_clearly_procedural_faq_question()`: regex detects 'how do I file', 'when does my policy expire', etc. → routed to FAQ service, not FNOL.
- 5. Keyword gate:** `_has_relevant_keywords()` + `_has_strong_keywords()` run on `_primary_message_text()` (latest non-quoted segment). Strong match can bypass LLM gate.
- 6. LLM FNOL gate:** gpt-4o-mini, temp=0, max_tokens=10. System: 'You classify P&C; insurance emails. Is this a First Notice of Loss? Answer only yes or no.' Bypassed if `FNOL_FILTER_ENABLED=false`.

4.3 Policy Number Extraction (Regex Patterns)

Function: `extract_policy_number()` in `backend/ingested_claims/service.py`

Keyword-anchored group 1:

```
poli?cy\s*(?:number|no\.?|#|id|ref(?:erence)?)\s*[:#\-=\s]\s*([A-Z0-9]{4,}?(?:[-/][A-Z0-9]+)*)
```

Claim number variant:

```
claim\s*(?:number|no\.?|#|id|ref(?:erence)?)\s*[:#\s]\s*([A-Z0-9]{4,}?(?:[-/][A-Z0-9]+)*)
```

Hash-prefix pattern:

```
#([A-Z]{1,4}\d{5,})
```

Structural – 2-letter prefix:

```
\b([A-Z]{2}\d{6,})\b
```

Structural – 3-letter prefix:

```
\b([A-Z]{3}\d{6,})\b
```

Structural – hyphenated:

```
\b([A-Z]{2,4}-\d{5,})\b
```

Fallback:

If none match → sender email address OR Message-ID display string used as placeholder

4.4 MIME Email Parsing

- Body: `_extract_body_text()` walks MIME tree; first text/plain part preferred; fallback to text/html with `_strip_html()`
- Headers: `_decode_header_value()` handles RFC2047 MIME-encoded words; `_format_address()` normalises From/To
- Attachments: collected as `{'filename', 'content_type', 'data': bytes}`; saved to `data/ingested-attachments/{claimId}/`
- SendGrid path: `extract_plain_body_from_rfc822()` parses raw RFC822 field when text/html are empty
- Thread quoted text: `_primary_message_text()` strips quoted reply blocks; filters run only on fresh content

5. Claim Extraction – Deep Dive

5.1 Email Field Extraction (gpt-4o)

Model	gpt-4o (env: OPENAI_MODEL, default gpt-4o)
Temperature	0.1
Max tokens	1500
System prompt	You are an expert insurance claims processor. Extract the following information from the FNOL email. Return ONLY valid JSON, no markdown.

Extracted Fields:

policyNumber	Policy number string from email body / subject
claimantName	Full name of the person filing the claim
contactEmail	Contact email of claimant
contactPhone	Contact phone number
lossDate	Date of loss/incident (ISO format preferred)
lossType	Enum: fire water auto theft liability medical other
lossLocation	Address or description of incident location
description	Full narrative description of the loss
vehicleInfo	Vehicle make / model / year / VIN (auto claims)
propertyAddress	Property address (home / commercial claims)
estimatedAmount	Claimant's estimated loss amount (numeric)
_confidence	Per-field confidence map (0.0 – 1.0) indicating extraction certainty

5.2 Document / Attachment Extraction (gpt-4o)

- Document types classified: PoliceReport | RepairEstimate | Invoice | MedicalRecord | IncidentReport | DamagePhoto | Other
- `_extract_schemas()`: returns type-specific field hints per document type
- Model: gpt-4o, temp=0.1, max_tokens=1500 – same config as email extraction
- Returns: documentType, keyFields{}, confidence

5.3 Vision / Image Analysis (gpt-4o multimodal)

- Input: image bytes base64-encoded → image_url data URL in message content list
- Model: gpt-4o (vision), max_tokens=2500
- Output fields: damage_severity, repair_cost_estimate, injury_indicators, detailed_summary, additional_observations
- Used for: damage photos, scanned documents with visual content

6. Policy Grounding & Decision Pack

6.1 Local Database Grounding (primary path)

- Input: policyNumber (from LLM extraction) OR sender email
- find_customer_by_email(): email → customer record from database/local_data/customers.json
- find_policy_by_number(): policyNumber → policy record from database/local_data/policies.json
- Policy active check: status == 'active' AND expiry_date >= today
- Coverage lookup: database/policy_grounding_mapping.json maps lossType × policy_type → applicable clause IDs
- Policy types handled: AUTO / HOME / COMMERCIAL
- Per-clause confidence scoring: base=1.0; deductions for inactive policy, near-expiry (≤30 days), lossType mismatch
- Returns: { groundingClauses[], coverageConfirmed, policyStatus, policyHolderInfo }
- Policy status flags: POLICY-EXPIRED | POLICY-INACTIVE | POLICY-NOT-FOUND | CUSTOMER-NOT-FOUND

6.2 ISO Clause Fallback (when local DB has no match)

- POLICY_CLAUSES: static library of ISO PAP / HO-3 / CGL-style clause texts in policy_clauses.py
- _infer_product_types(): policy number prefix (AC→auto, HO→home, CL→commercial)
- _infer_loss_types(): regex on lossType field + description narrative
- _compute_similarity(): keyword overlap between clause text and combined claim description
- Thresholds: CONFIDENCE_THRESHOLD_HIGH=0.8, MEDIUM=0.6; clauses with score < 0.6 excluded
- Max 6 clauses returned; sorted by descending similarity score

6.3 Decision Pack Assembly

- build_decision_pack(claim_id, extraction_result) in backend/decision/service.py
- Merges all extracted fields; resolves policy via LLM result → DB lookup → fallback
- Runs local grounding first; if empty → ISO clause fallback
- Derives coverage_confirmed: True if grounding returned clauses + policy active
- Auto-reject flags: POLICY-EXPIRED / INACTIVE / NOT-FOUND / CUSTOMER-NOT-FOUND → triggers desk rejection in orchestrator
- Decision pack contains: claimId, timestamp, processingStatus, extractedFields (with _confidence), evidence[], documentAnalysis[], groundingClauses[], coverageConfirmed, policyHolderInfo, auditMetadata
- No LLM used in decision/grounding; entirely rule-based and DB-driven

7. Notifications & FAQ Resolution

7.1 Automated Email Notifications (claim_notification/service.py)

Notification	Trigger	Toggle Env Var	Content
FNOL Receipt Ack	After IMAP/SendGrid ingest	FNOL_RECEIPT_ACK_EMAIL_ENABLED	'We received your claim, reference #{claimId}'
Under Review	After save_processed_claim() (non-rejected)	CLAIM_UNDER_REVIEW_EMAIL_ENABLED	Claim details, policy holder info, next steps
Desk Rejection	Orchestrator detects expired/invalid policy	DESK_REJECTION_EMAIL_ENABLED	Rejection reason, policy status, appeal instructions

All notifications use smtplib SMTP with In-Reply-To / References headers to thread into original email.

7.2 FAQ Resolution (faq_resolution/service.py)

- Triggered: email classified as non-FNOL by email_ingestion or directly via source=faq inbox
- Dedup: check faq-answered-ids.json; skip if message_id already answered
- _is_faq_query(): regex keyword patterns first; then gpt-4o-mini 'FAQ or CLAIM?' classification
- Customer data intent: _classify_customer_data_intent() → detect 'what is my deductible / policy limit / status' → answer from local JSON DB
- FAQ answer: _find_faq_answer() → gpt-4o-mini receives full data/FAQ.csv; prompted to pick 'Q1', 'Q2', ... index; extract answer
- Keyword fallback: if no API key → overlap score between question text and FAQ rows
- _send_faq_response_email(): SMTP threaded reply (In-Reply-To / References set)
- Persist: message_id stored to faq-answered-ids.json to prevent duplicate replies

8. FastAPI API Layer & Frontend

8.1 FastAPI REST Endpoints

Method	Path	Purpose
GET	/health	Liveness probe
GET	/	Root info endpoint
POST	/api/process-claim	Run full orchestration pipeline on ingested claim
GET	/api/ingested-claims	List all ingested claims; query: full=bool, source=faq
GET	/api/ingested-claims/{claim_id}	Get single ingested claim by ID
POST	/api/ingested-claims/clear	Wipe all ingested claims (dev/demo reset)
POST	/api/claims/clear	Wipe all processed claims (dev/demo reset)
GET	/api/claims	List all processed claims
POST	/api/claims	Create / update processed claim record
GET	/api/claims/{claim_id}	Get single processed claim by ID
GET	/api/dashboard/kpis	Return KPI aggregations for dashboard charts
POST	/api/sync-inbox	Trigger IMAP inbox sync (runs email_ingestion pipeline)

8.2 Frontend Components (Next.js)

app/page.tsx	Stage router: home → review → decision → dashboard; auth gate; FAQ page toggle
HomePage.tsx	Ingested claim inbox; IMAP sync via POST /api/sync-inbox; process claim via POST /api/process-claim
ReviewPage.tsx	Displays extracted fields + evidence list for adjuster review
DecisionPage.tsx	Shows decision pack: policy grounding, coverage confirmed, policy holder info
DashboardPage.tsx	KPI charts via GET /api/dashboard/kpis; Recharts visualisations
FAQPage.tsx	FAQ-sourced inbox (source=faq query param to ingested-claims endpoint)
Header.tsx	Navigation bar between pages
ClaimSummaryBar.tsx	Persistent claim summary strip across all stages
lib/backend.ts	runPython() helper for legacy webhook spawning
lib/agents/orchestrator.ts	LangGraph-style client-side orchestrator (present but NOT used in main claim flow – FastAPI path is active)
app/api/webhooks/sendgrid-inbound/router.ts	Receives SendGrid inbound-parse POST; forwards to FastAPI ingested-claims endpoint

9. Data Stores & Environment Variables

9.1 Data Storage (JSON / CSV – no SQL at runtime)

data/ingested-claims.json	Ingested claims	All raw ingested emails (FNOL + FAQ)
data/ingested-attachments/{claimId}/	Attachment files	Binary/text attachments per claim
data/faq-answered-ids.json	FAQ dedup list	Message IDs of already-answered FAQ emails
data/processed-claims/{claimId}.json	Processed claim	Full decision pack per claim
data/processed-claims/claims-index.json	Claims index	Lightweight index of all processed claims
data/processed-claims/claims-history.csv	Claims CSV	Append-only CSV for audit/analytics
database/local_data/customers.json	Customer DB	Customer records (name, email, address)
database/local_data/policies.json	Policy DB	Policy records (type, status, limits, expiry)
database/local_data/policy_details.json	Policy details DB	Detailed clause and coverage info
database/policy_grounding_mapping.json	Coverage mapping	lossType × policy_type → clause IDs
data/FAQ.csv	FAQ dataset	Question-answer pairs for FAQ resolution

9.2 Environment Variables

OPENAI_API_KEY	OpenAI	Required for LLM calls (gpt-4o / gpt-4o-mini)
OPENAI_MODEL	OpenAI	Default: gpt-4o (extraction); gpt-4o-mini (FNOL gate, FAQ)
IMAP_HOST	IMAP	Default: imap.gmail.com
IMAP_PORT	IMAP	Default: 993
SENDER_EMAIL / IMAP_USER	IMAP	Inbox email address
EMAIL_PASSWORD / IMAP_PASSWORD	IMAP	App password (spaces stripped)
IMAP_MAILBOX	IMAP	Default: INBOX
IMAP_SYNC_INCLUDE_READ	IMAP	Default: true – fetch ALL, not just UNSEEN
IMAP_SYNC_MAX_EMAILS	IMAP	Default: 0 (unlimited)
IMAP_GMAIL_TRY_ALL_MAIL	IMAP	Default: true – try [Gmail]/All Mail first
IMAP_SSL_VERIFY	IMAP	Default: false – skip cert verification
FNOL_FILTER_ENABLED	FNOL Gate	Default: true – use LLM FNOL classification
FNOL_VERBOSE_SKIP_LOGS	FNOL Gate	Verbose logging for skipped emails
FNOL_RECEIPT_ACK_EMAIL_ENABLED	Notifications	Default: on – send FNOL ack
CLAIM_UNDER_REVIEW_EMAIL_ENABLED	Notifications	Default: on – send under-review email
DESK_REJECTION_EMAIL_ENABLED	Notifications	Default: on – send desk-rejection email
SMTP_HOST	SMTP	Outbound SMTP host
SMTP_PORT	SMTP	Outbound SMTP port
SMTP_SECURE	SMTP	TLS/SSL flag
API_HOST	FastAPI	Default: 0.0.0.0
API_PORT	FastAPI	Default: 8020
API_RELOAD	FastAPI	Uvicorn hot reload toggle
CORS_ORIGINS	FastAPI	Default: * (allow all origins)

LOCAL_DATA_DIR	Policy DB	Default: database/local_data
POLICY_GROUNDING_MAPPING_FILE	Policy DB	Default: database/policy_grounding_mapping.json
FAQ_CSV_FILE	FAQ	Default: data/FAQ.csv
DATA_DIR / INGESTED_DIR / PROCESSED_CLAIMS_DIR	Paths	Data directory overrides
ENV_FILE	Config	Path to .env file
NEXT_PUBLIC_API_URL	Frontend	FastAPI base URL (default: http://localhost:8020)
NEXT_PUBLIC_OPENAI_API_KEY	Frontend	OpenAI key for client-side SDK (legacy flows)